

Aprendizaje de Máquina

Sesión 10: Agrupamiento basado en distancia.

Angel Rodríguez

DBSCAN: anomalías

Una observación se considera una **anomalía** si:

- no es una observación núcleo, y
- no pertenece al vecindario de ninguna observación núcleo.

En Scikit-Learn, estas observaciones reciben la etiqueta

-1 .

Esto permite que DBSCAN pueda realizar simultáneamente:

- clustering,
- detección de observaciones aisladas.

DBSCAN en Scikit-Learn

Utilicemos el conjunto de datos `make_moons`:

```
from sklearn.cluster import DBSCAN
from sklearn.datasets import make_moons
import matplotlib.pyplot as plt

X, y = make_moons(
    n_samples=1000,
    noise=0.05,
    random_state=42
)

plt.scatter(X[:,0],X[:,1],s=1)
plt.show()

dbscan = DBSCAN(
    eps=0.05,
    min_samples=5
)

dbscan.fit(X)
```

Etiquetas encontradas por DBSCAN

Después de entrenar:

dbscan.labels_

podemos obtener, por ejemplo:

```
array([0, 2, -1, -1, 1, 0, ..., 3])
```

Los valores

 $0, 1, 2, \dots$

representan los clusters encontrados.

La etiqueta

-1

representa una anomalía.

Observaciones núcleo

Scikit-Learn guarda los índices de las observaciones núcleo en:

dbscan.core_sample_indices_

Y las observaciones núcleo se encuentran en:

dbscan.components_

Por ejemplo:

components_

contiene las coordenadas de las observaciones que DBSCAN identificó como pertenecientes a regiones densas.

El efecto de ε

El valor de ε tiene un efecto muy importante.

Si

$$\varepsilon$$

es demasiado pequeño:

- aparecen muchos clusters pequeños,
- muchas observaciones pueden clasificarse como anomalías.

Si aumentamos ε :

- los vecindarios se hacen más grandes,
- más observaciones pueden conectarse,
- los clusters pueden fusionarse.

Función para graficar el resultado:

```
def plot_dbscan(dbscan, X, size, show_xlabels=True, show_ylabels=True):
    core_mask = np.zeros_like(dbscan.labels_, dtype=bool)
    core_mask[dbscan.core_sample_indices_] = True
    anomalies_mask = dbscan.labels_ == -1
    non_core_mask = ~(core_mask | anomalies_mask)

    cores = dbscan.components_
    anomalies = X[anomalies_mask]
    non_cores = X[non_core_mask]

    plt.scatter(cores[:, 0], cores[:, 1],
                c=dbscan.labels_[core_mask], marker='o', s=size, cmap="Paired")
    plt.scatter(cores[:, 0], cores[:, 1], marker='*', s=20,
                c=dbscan.labels_[core_mask])
    plt.scatter(anomalies[:, 0], anomalies[:, 1],
                c="r", marker="x", s=100)
    plt.scatter(non_cores[:, 0], non_cores[:, 1],
                c=dbscan.labels_[non_core_mask], marker=".")
    if show_xlabels:
        plt.xlabel("$x_1$")
    else:
        plt.tick_params(labelbottom=False)
    if show_ylabels:
        plt.ylabel("$x_2$", rotation=0)
    else:
        plt.tick_params(labelleft=False)
    plt.title(f"eps={dbscan.eps:.2f}, min_samples={dbscan.min_samples}")
    plt.grid()
    plt.gca().set_axisbelow(True)
```

Graficar el resultado

```
import matplotlib.pyplot as plt
import numpy as np

plot_dbscan(dbscan, X, size=100)
plt.show()
```

Ejemplo: cambiar ε

Con $\varepsilon = 0,05$, se encuentran muchos clusters y muchas anomalías.

Utilicemos:

```
dbscan2 = DBSCAN(eps=0.2)
dbscan2.fit(X)
plot_dbscan(dbscan2, X, size=100)
plt.show()
```

```
dbscan3 = DBSCAN(eps=1)
dbscan3.fit(X)
plot_dbscan(dbscan3, X, size=50)
plt.show()
```

para ver qué pasa.

Los dos hiperparámetros de DBSCAN

DBSCAN tiene dos hiperparámetros principales:

ϵ y `min_samples`.

ϵ :

determina el radio del vecindario.

`min_samples`:

determina cuántas observaciones deben existir dentro del vecindario para considerar una observación como núcleo.

Ambos parámetros controlan lo que DBSCAN considera una **región suficientemente densa**.

DBSCAN vs. K-means

	K-means	DBSCAN
Número de clusters	Se especifica	Se determina
Centroide	Sí	No
Formas arbitrarias	Limitado	Sí
Anomalías	No	Sí
Basado en	Distancia al centro	Densidad

DBSCAN puede funcionar especialmente bien cuando los clusters tienen formas complejas y están separados por regiones de baja densidad.

Limitación: diferentes densidades

DBSCAN funciona bien cuando los clusters están separados por regiones de baja densidad.

Sin embargo, puede tener dificultades cuando:

- los clusters tienen densidades muy diferentes,
- no existe una región suficientemente poco densa entre dos clusters.

Un único valor de ε puede ser adecuado para un cluster, pero inadecuado para otro.

Ventajas y desventajas de DBSCAN

Ventajas

- No requiere especificar el número de clusters.
- Puede encontrar clusters con formas arbitrarias.
- Es robusto frente a observaciones aisladas.
- Solo tiene dos hiperparámetros principales.

Desventajas

- Puede fallar cuando las densidades de los clusters son muy diferentes.
- Puede tener problemas si los clusters no están separados por regiones de baja densidad.
- No escala bien a conjuntos de datos muy grandes.

¿Cómo predecimos nuevas observaciones?

Una particularidad de la implementación de DBSCAN en Scikit-Learn es que no tiene un método

`predict()`.

DBSCAN encuentra los clusters de los datos utilizados durante el entrenamiento, pero no define directamente cómo asignar una observación nueva.

Si necesitamos hacerlo, podemos entrenar posteriormente otro clasificador utilizando los clusters encontrados por DBSCAN.

Ejemplo: utilizar KNN después de DBSCAN

Podemos entrenar un clasificador KNN utilizando las observaciones núcleo:

```
from sklearn.neighbors import KNeighborsClassifier  
  
knn = KNeighborsClassifier(n_neighbors=50)  
  
knn.fit(dbscan2.components_, dbscan2.labels_[dbscan2.core_sample_indices_  
    ])
```

Ahora KNN puede utilizar los clusters encontrados por DBSCAN para clasificar nuevas observaciones.

Graficar regiones de decisión (función vista anteriormente)

```
def plot_data(X):
    plt.plot(X[:, 0], X[:, 1], 'k.', markersize=2)

def plot_decision_boundaries(clusterer, X, resolution=1000, show_centroids=True,
                             show_xlabels=True, show_ylabels=True):
    mins = X.min(axis=0) - 0.1
    maxs = X.max(axis=0) + 0.1
    xx, yy = np.meshgrid(np.linspace(mins[0], maxs[0], resolution), np.linspace(mins[1], maxs[1],
                                         resolution))
    Z = clusterer.predict(np.c_[xx.ravel(), yy.ravel()])
    Z = Z.reshape(xx.shape)

    plt.contourf(Z, extent=(mins[0], maxs[0], mins[1], maxs[1]), cmap="Pastel2")
    plt.contour(Z, extent=(mins[0], maxs[0], mins[1], maxs[1]), linewidths=1, colors='k')
    plot_data(X)
    if show_centroids:
        plot_centroids(clusterer.cluster_centers_)

    if show_xlabels:
        plt.xlabel("$x_1$")
    else:
        plt.tick_params(labelbottom=False)
    if show_ylabels:
        plt.ylabel("$x_2$", rotation=0)
    else:
        plt.tick_params(labelleft=False)
```

Graficar regiones de decisión (función vista anteriormente)

```
def plot_centroids(centroids, weights=None, circle_color='w', cross_color='k'):  
    if weights is not None:  
        centroids = centroids[weights > weights.max() / 10]  
    plt.scatter(centroids[:, 0], centroids[:, 1],  
                marker='o', s=35, linewidths=8,  
                color=circle_color, zorder=10, alpha=0.9)  
    plt.scatter(centroids[:, 0], centroids[:, 1],  
                marker='x', s=2, linewidths=12,  
                color=cross_color, zorder=11, alpha=1)
```

Graficar datos nuevos

Díganme 4 puntos $X_i = (x_i, y_i)$, con $x_i \in [-1, 1, 2, 1]$ y $y_i \in [-0.6, 1, 1]$.

```
X_new = np.array([[-0.5, 0], [0, 0.5], [1, -0.1], [2, 1]])
plt.figure(figsize=(6, 3))
plot_decision_boundaries(knn, X, show_centroids=False)
plt.scatter(X_new[:, 0], X_new[:, 1], c="b", marker="+", s=200, zorder
            =10)

plt.show()

labels = knn.predict(X_new)
print(labels)
```

¿Por qué mezclas Gaussianas?

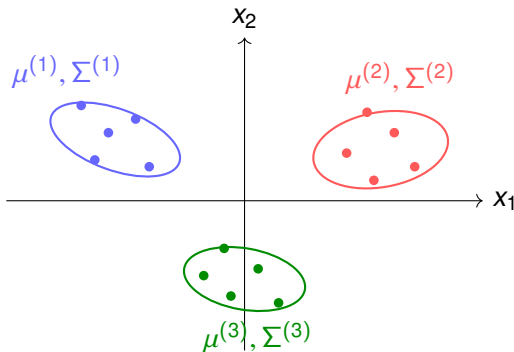
- Un **Gaussian Mixture Model** (GMM) es un modelo probabilístico.
- Supone que los datos provienen de una mezcla de varias distribuciones normales.
- Sirve para:
 - estimación de densidad,
 - clustering,
 - detección de anomalías.

Idea visual

Cada componente Gaussiana genera una nube elipsoidal de puntos.

Suposición generativa

Dado un conjunto de datos X , suponemos que cada instancia fue generada por uno de k componentes Gaussianos.



Cada componente puede tener diferente **forma, tamaño, densidad y orientación**.

Proceso probabilístico

Para cada instancia $\mathbf{x}^{(i)}$:

1. Se elige un componente $z^{(i)} \in \{1, \dots, k\}$ con pesos

$$\phi^{(1)}, \dots, \phi^{(k)}, \quad \sum_{j=1}^k \phi^{(j)} = 1.$$

2. Si $z^{(i)} = j$, entonces

$$\mathbf{x}^{(i)} \sim \mathcal{N}(\boldsymbol{\mu}^{(j)}, \boldsymbol{\Sigma}^{(j)}).$$

Parámetros que queremos estimar

$$\boldsymbol{\phi}, \quad \boldsymbol{\mu}^{(1)}, \dots, \boldsymbol{\mu}^{(k)}, \quad \boldsymbol{\Sigma}^{(1)}, \dots, \boldsymbol{\Sigma}^{(k)}.$$

Generar datos para ejemplo:

```
from sklearn.datasets import make_blobs
import numpy as np
import matplotlib.pyplot as plt

X1, y1 = make_blobs(n_samples=1000, centers=((4, -4), (0, 0)),
                    random_state=42)
X1 = X1.dot(np.array([[0.374, 0.95], [0.732, 0.598]]))
X2, y2 = make_blobs(n_samples=250, centers=1, random_state=42)
X2 = X2 + [6, -8]
X = np.r_[X1, X2]
y = np.r_[y1, y2]

C1 = [-1.432, 1.408]
C2 = [0, 0]
C3 = [3.491, 1.014]

plt.scatter(X[:, 0], X[:, 1], s=1)
plt.scatter([C1[0], C2[0], C3[0]], [C1[1], C2[1], C3[1]], color="k", marker="x")
plt.xlabel("$x_1$")
plt.ylabel("$x_2$")
plt.show()
```

Matrices de covarianza de los blobs

Inicialmente, `make_blobs` genera cada grupo con $\Sigma = I$.
Para los dos primeros blobs se aplica la transformación

$$\mathbf{x}' = \mathbf{x}\mathbf{A}, \quad \mathbf{A} = \begin{bmatrix} 0,374 & 0,95 \\ 0,732 & 0,598 \end{bmatrix}.$$

Por lo tanto,

$$\Sigma_1 = \Sigma_2 = \mathbf{A}^\top \mathbf{A} = \begin{bmatrix} 0,676 & 0,793 \\ 0,793 & 1,260 \end{bmatrix}.$$

El tercer blob únicamente se traslada, por lo que su covarianza no cambia:

$$\Sigma_3 = I = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}.$$

Ajuste con Scikit-Learn

```
from sklearn.mixture import GaussianMixture

gm = GaussianMixture(n_components=3, n_init=10, random_state=42)
gm.fit(X)
```

Después de ajustar el modelo podemos inspeccionar:

- `gm.weights_`: pesos $\phi^{(j)}$,
- `gm.means_`: medias $\mu^{(j)}$,
- `gm.covariances_`: matrices de covarianza $\Sigma^{(j)}$.

Nota

En `GaussianMixture` se debe indicar por adelantado el número de componentes k .

Algoritmo EM: Expectation–Maximization

El modelo se ajusta usando el algoritmo EM, que alterna dos pasos hasta converger.

E-step

Estima, para cada instancia, la probabilidad de pertenecer a cada componente.

M-step

Actualiza pesos, medias y covarianzas usando esas probabilidades como pesos.

$$\gamma_{ij} = P(z^{(i)} = j \mid \mathbf{x}^{(i)})$$

Estas probabilidades se llaman **responsabilidades**.

¿Suenan parecido?

EM para Gaussian Mixtures: idea central

EM es como K-means, pero probabilístico.

	K-means	GMM con EM
Asignación	dura (uno a uno)	suave (probabilidades)
Parámetros	centroides	medias, covarianzas y pesos

E-step: calcular qué tanto pertenece cada punto a cada cluster.

$$\gamma_{ij} = P(z_i = j \mid x_i) = \frac{\phi_j \mathcal{N}(x_i \mid \mu_j, \Sigma_j)}{\sum_{\ell=1}^k \phi_{\ell} \mathcal{N}(x_i \mid \mu_{\ell}, \Sigma_{\ell})}$$

γ_{ij} = responsabilidad del cluster j sobre el punto i .

EM para Gaussian Mixtures: actualizar clusters

M-step: actualizar los parámetros usando las responsabilidades.

Primero calculamos el “tamaño efectivo” del cluster:

$$N_j = \sum_{i=1}^m \gamma_{ij}$$

EM para Gaussian Mixtures: actualizar clusters

M-step: actualizar los parámetros usando las responsabilidades.

Primero calculamos el “tamaño efectivo” del cluster:

$$N_j = \sum_{i=1}^m \gamma_{ij}$$

Actualizamos el peso del cluster:

$$\phi_j = \frac{N_j}{m}$$

EM para Gaussian Mixtures: actualizar clusters

M-step: actualizar los parámetros usando las responsabilidades.

Primero calculamos el “tamaño efectivo” del cluster:

$$N_j = \sum_{i=1}^m \gamma_{ij}$$

Actualizamos el peso del cluster:

$$\phi_j = \frac{N_j}{m}$$

Actualizamos la media:

$$\mu_j = \frac{1}{N_j} \sum_{i=1}^m \gamma_{ij} x_i$$

EM para Gaussian Mixtures: actualizar clusters

M-step: actualizar los parámetros usando las responsabilidades.

Primero calculamos el “tamaño efectivo” del cluster:

$$N_j = \sum_{i=1}^m \gamma_{ij}$$

Actualizamos el peso del cluster:

$$\phi_j = \frac{N_j}{m}$$

Actualizamos la media:

$$\mu_j = \frac{1}{N_j} \sum_{i=1}^m \gamma_{ij} x_i$$

Actualizamos la covarianza:

$$\Sigma_j = \frac{1}{N_j} \sum_{i=1}^m \gamma_{ij} (x_i - \mu_j)(x_i - \mu_j)^T$$

Resumen: K-means recalcula centroides; EM recalcula medias, formas y pesos.

EM como generalización de k -means

	k -means	GMM con EM
Asignación	dura	suave
Centro	centroides	medias $\mu^{(j)}$
Forma	implícitamente circular	elipsoidal
Tamaño/densidad	similar	puede variar
Parámetros	centros	pesos, medias, covarianzas

Cuidado

Como k -means, EM puede converger a soluciones pobres. Por eso conviene usar varios reinicios, por ejemplo `n_init=10`.

Clustering duro y suave

Una vez ajustado, el GMM puede asignar clusters de dos formas.

Hard clustering

Asigna cada punto al componente más probable.

```
gm.predict(X)
```

Soft clustering

Devuelve la probabilidad de pertenecer a cada componente.

```
gm.predict_proba(X)
```

La asignación suave es útil cuando los clusters se traslapan.

Modelo generativo y densidad

Un GMM no solo agrupa: también describe una distribución de probabilidad.

- Puede generar nuevas instancias:

```
X_new, y_new = gm.sample(6)
```

- Puede estimar la log-densidad (mayor = más denso) en una ubicación:

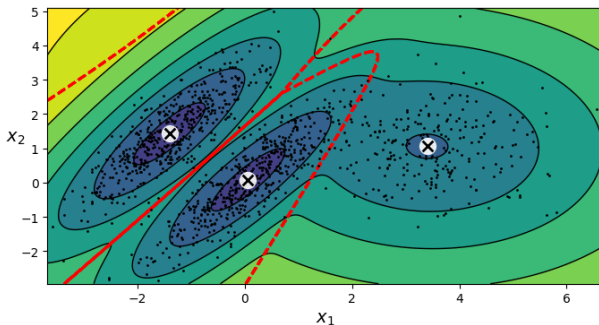
```
gm.score_samples(X)
```

Importante

`score_samples()` devuelve el logaritmo de la función de densidad. Su exponencial es una densidad, no una probabilidad directa.

Densidad, fronteras y contornos

- Las medias indican el centro de cada componente.
- Las fronteras de decisión separan regiones donde domina cada componente.
- Los contornos de densidad muestran zonas más o menos probables.



En datos reales, el reto crece si hay muchas dimensiones, muchos clusters o pocas observaciones.

Restricciones en las covarianzas

Para simplificar el modelo se puede restringir la forma de las matrices de covarianza con `covariance_type`.

covariance_type	Restricción geométrica
spherical	clusters esféricos, con varianzas distintas
diag	elipses alineadas con los ejes
tied	todos comparten la misma covarianza
full (default)	cada cluster tiene covarianza libre

Trade-off

Más flexibilidad puede mejorar el ajuste, pero aumenta el número de parámetros y el costo computacional.

Costo computacional

Sea:

- m : número de instancias,
- n : número de dimensiones,
- k : número de componentes.

Tipo	Complejidad aproximada	Comentario
spherical, diag	$O(kmn)$	escala mejor
tied, full	$O(kmn^2 + kn^3)$	costoso si n es grande

Consecuencia práctica

En alta dimensión puede ser necesario reducir dimensionalidad o usar restricciones en la covarianza.

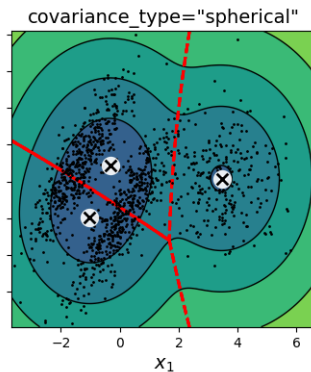
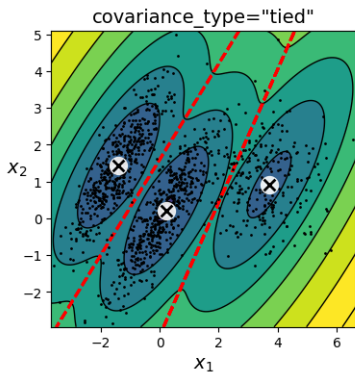
Tipos de covarianza

```
gm_full = GaussianMixture(n_components=3, n_init=10, covariance_type="full",
                           random_state=42)
gm_tied = GaussianMixture(n_components=3, n_init=10, covariance_type="tied",
                           random_state=42)
gm_spherical = GaussianMixture(n_components=3, n_init=10, covariance_type="spherical",
                                random_state=42)
gm_diag = GaussianMixture(n_components=3, n_init=10, covariance_type="diag",
                           random_state=42)

gm_full.fit(X)
gm_tied.fit(X)
gm_spherical.fit(X)
gm_diag.fit(X)

plot_gaussian_mixture(gm_full, X)
plot_gaussian_mixture(gm_tied, X)
plot_gaussian_mixture(gm_spherical, X)
plot_gaussian_mixture(gm_diag, X)
```

Gráficas



GMM para detección de anomalías

La idea es simple: puntos en regiones de baja densidad pueden considerarse anomalías.

```
densities = gm.score_samples(X)
density_threshold = np.percentile(densities, 2)
anomalies = X[densities < density_threshold]

plt.figure(figsize=(8, 4))
plot_gaussian_mixture(gm, X)
plt.scatter(anomalies[:, 0], anomalies[:, 1], color='r', marker='*')
plt.ylim(top=5.1)
plt.show()
```

- El percentil define el umbral de densidad.
- Si esperamos cerca de 2 % de defectos, podemos usar el percentil 2.
- Ajustar el umbral cambia el balance entre falsos positivos y falsos negativos.

Anomalía vs. novedad

Detección de anomalías

El conjunto de entrenamiento puede contener algunos outliers. El método puede usarse para identificarlos y limpiar los datos.

Detección de novedades

El modelo se entrena con datos “limpios”. Luego se buscan observaciones nuevas que no parezcan venir de la distribución aprendida.

Resumen: un GMM aprende pesos, medias y covarianzas; permite clustering suave, estimación de densidad, generación de datos y detección de anomalías.

Selección del número de centros

- En GaussianMixture, al igual que en k -means, debemos especificar el número de clusters k .
- Para k -means suelen usarse métricas como inercia.
- En mezclas gaussianas, estas métricas pueden ser poco confiables cuando los clusters no son esféricos o tienen tamaños distintos.
- Por eso se usan criterios de información: **BIC** y **AIC**.

Idea general: balance ajuste vs. complejidad

Objetivo

Elegir el modelo que explica bien los datos, pero sin usar demasiados parámetros.

Idea general: balance ajuste vs. complejidad

Objetivo

Elegir el modelo que explica bien los datos, pero sin usar demasiados parámetros.

- Más clusters $\Rightarrow$ más parámetros.
- Más parámetros $\Rightarrow$ mayor riesgo de sobreajuste.
- BIC y AIC penalizan la complejidad y recompensan el buen ajuste.

Idea general: balance ajuste vs. complejidad

Objetivo

Elegir el modelo que explica bien los datos, pero sin usar demasiados parámetros.

- Más clusters $\Rightarrow$ más parámetros.
- Más parámetros $\Rightarrow$ mayor riesgo de sobreajuste.
- BIC y AIC penalizan la complejidad y recompensan el buen ajuste.

Regla práctica

Se elige el modelo con **menor** BIC o AIC.

Crterios BIC y AIC

Definiciones

$$BIC = \log(m) p - 2 \log(\mathcal{L})$$

$$AIC = 2p - 2 \log(\mathcal{L})$$

Crterios BIC y AIC

Definiciones

$$BIC = \log(m) p - 2 \log(\mathcal{L})$$

$$AIC = 2p - 2 \log(\mathcal{L})$$

- m : número de instancias.
- p : número de parámetros aprendidos por el modelo.
- $\mathcal{L}$: valor máximo de la función de verosimilitud.

Comparación entre BIC y AIC

BIC

- Penaliza la complejidad con $\log(m)p$.
- Suele preferir modelos más simples.
- La penalización crece con el tamaño del dataset.

AIC

- Penaliza la complejidad con $2p$.
- Puede elegir modelos más flexibles.
- Suele ajustar mejor, pero con mayor complejidad.

Comparación entre BIC y AIC

BIC

- Penaliza la complejidad con $\log(m)p$.
- Suele preferir modelos más simples.
- La penalización crece con el tamaño del dataset.

AIC

- Penaliza la complejidad con $2p$.
- Puede elegir modelos más flexibles.
- Suele ajustar mejor, pero con mayor complejidad.

Cuando difieren, BIC tiende a seleccionar un modelo más simple que AIC.

Probabilidad vs. verosimilitud

Probabilidad

Con parámetros fijos θ , mide qué tan plausible es un resultado futuro x :

$$f(x; \theta)$$

Probabilidad vs. verosimilitud

Probabilidad

Con parámetros fijos θ , mide qué tan plausible es un resultado futuro x :

$$f(x; \theta)$$

Verosimilitud

Con datos observados fijos, mide qué tan plausibles son los parámetros:

$$\mathcal{L}(\theta | x) = f(x; \theta)$$

Probabilidad vs. verosimilitud

Probabilidad

Con parámetros fijos θ , mide qué tan plausible es un resultado futuro x :

$$f(x; \theta)$$

Verosimilitud

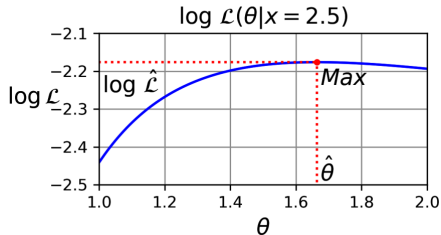
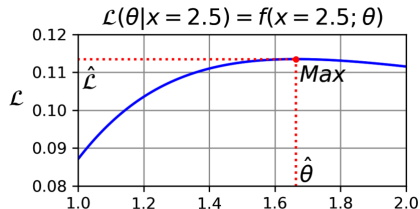
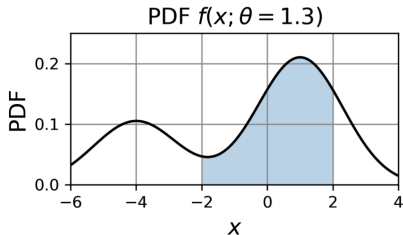
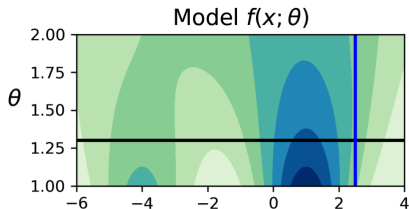
Con datos observados fijos, mide qué tan plausibles son los parámetros:

$$\mathcal{L}(\theta | x) = f(x; \theta)$$

Diferencia clave

La PDF es función de x ; la verosimilitud es función de θ .

Probabilidad vs. verosimilitud



Estimación por máxima verosimilitud

- Dado un conjunto de datos X , buscamos los parámetros más plausibles.

Estimación por máxima verosimilitud

- Dado un conjunto de datos X , buscamos los parámetros más plausibles.
- Esto se hace maximizando la verosimilitud:

$$\hat{\theta}_{MLE} = \arg \max_{\theta} \mathcal{L}(\theta | X).$$

Estimación por máxima verosimilitud

- Dado un conjunto de datos X , buscamos los parámetros más plausibles.
- Esto se hace maximizando la verosimilitud:

$$\hat{\theta}_{MLE} = \arg \max_{\theta} \mathcal{L}(\theta | X).$$

- Si incorporamos una distribución previa $g(\theta)$, maximizamos:

$$\mathcal{L}(\theta | X)g(\theta).$$

Estimación por máxima verosimilitud

- Dado un conjunto de datos X , buscamos los parámetros más plausibles.
- Esto se hace maximizando la verosimilitud:

$$\hat{\theta}_{MLE} = \arg \max_{\theta} \mathcal{L}(\theta | X).$$

- Si incorporamos una distribución previa $g(\theta)$, maximizamos:

$$\mathcal{L}(\theta | X)g(\theta).$$

- Esto se llama **MAP**: máxima estimación a posteriori.

MLE vs. MAP

MLE

$$\arg \max_{\theta} \mathcal{L}(\theta | X)$$

- Usa solo los datos observados.
- Busca el mejor ajuste posible.

MAP

$$\arg \max_{\theta} \mathcal{L}(\theta | X)g(\theta)$$

- Usa datos y conocimiento previo.
- Puede verse como una versión regularizada de MLE.

Por qué usar log-verosimilitud

- Maximizar $\mathcal{L}$ es equivalente a maximizar $\log \mathcal{L}$, porque el logaritmo es estrictamente creciente.

Por qué usar log-verosimilitud

- Maximizar $\mathcal{L}$ es equivalente a maximizar $\log \mathcal{L}$, porque el logaritmo es estrictamente creciente.
- Con observaciones independientes:

$$\mathcal{L}(\theta | X) = \prod_{i=1}^m \mathcal{L}(\theta | x^{(i)}).$$

Por qué usar log-verosimilitud

- Maximizar $\mathcal{L}$ es equivalente a maximizar $\log \mathcal{L}$, porque el logaritmo es estrictamente creciente.
- Con observaciones independientes:

$$\mathcal{L}(\theta | X) = \prod_{i=1}^m \mathcal{L}(\theta | x^{(i)}).$$

- Al tomar logaritmos:

$$\log \mathcal{L}(\theta | X) = \sum_{i=1}^m \log \mathcal{L}(\theta | x^{(i)}).$$

Por qué usar log-verosimilitud

- Maximizar $\mathcal{L}$ es equivalente a maximizar $\log \mathcal{L}$, porque el logaritmo es estrictamente creciente.
- Con observaciones independientes:

$$\mathcal{L}(\theta | X) = \prod_{i=1}^m \mathcal{L}(\theta | x^{(i)}).$$

- Al tomar logaritmos:

$$\log \mathcal{L}(\theta | X) = \sum_{i=1}^m \log \mathcal{L}(\theta | x^{(i)}).$$

- Esto es más estable y más fácil de optimizar.

Verosimilitud en un Gaussian Mixture Model

Para una observación $\mathbf{x}_i$, la densidad bajo un GMM con k componentes es:

$$p(\mathbf{x}_i; \theta) = \sum_{j=1}^k \phi_j \mathcal{N}(\mathbf{x}_i | \mu_j, \Sigma_j)$$

Verosimilitud en un Gaussian Mixture Model

Para una observación $\mathbf{x}_i$, la densidad bajo un GMM con k componentes es:

$$p(\mathbf{x}_i; \theta) = \sum_{j=1}^k \phi_j \mathcal{N}(\mathbf{x}_i | \mu_j, \Sigma_j)$$

Si las m observaciones son independientes, la función de verosimilitud es:

$$\mathcal{L}(\theta; X) = \prod_{i=1}^m \left[\sum_{j=1}^k \phi_j \mathcal{N}(\mathbf{x}_i | \mu_j, \Sigma_j) \right]$$

Verosimilitud en un Gaussian Mixture Model

Para una observación $\mathbf{x}_i$, la densidad bajo un GMM con k componentes es:

$$p(\mathbf{x}_i; \theta) = \sum_{j=1}^k \phi_j \mathcal{N}(\mathbf{x}_i | \mu_j, \Sigma_j)$$

Si las m observaciones son independientes, la función de verosimilitud es:

$$\mathcal{L}(\theta; X) = \prod_{i=1}^m \left[\sum_{j=1}^k \phi_j \mathcal{N}(\mathbf{x}_i | \mu_j, \Sigma_j) \right]$$

En la práctica se maximiza la **log-verosimilitud**:

$$\log \mathcal{L}(\theta; X) = \sum_{i=1}^m \log \left[\sum_{j=1}^k \phi_j \mathcal{N}(\mathbf{x}_i | \mu_j, \Sigma_j) \right]$$

Idea

Para cada punto se suman las contribuciones de todas las gaussianas; EM busca los parámetros θ que maximizan esta verosimilitud.

Calcular BIC y AIC en Scikit-Learn

Después de ajustar una mezcla gaussiana:

```
from sklearn.mixture import GaussianMixture

gm = GaussianMixture(n_components=3, n_init=10,
                     random_state=42)
gm.fit(X)

bic = gm.bic(X)
aic = gm.aic(X)
```

Calcular BIC y AIC en Scikit-Learn

Después de ajustar una mezcla gaussiana:

```
from sklearn.mixture import GaussianMixture

gm = GaussianMixture(n_components=3, n_init=10,
                     random_state=42)
gm.fit(X)

bic = gm.bic(X)
aic = gm.aic(X)
```

- `bic(X)` calcula el criterio BIC.
- `aic(X)` calcula el criterio AIC.
- Se repite para distintos valores de k y se elige el mínimo.

Procedimiento práctico para elegir k

1. Elegir un rango de valores candidatos: $k = 1, 2, \dots, K$.

Procedimiento práctico para elegir k

1. Elegir un rango de valores candidatos: $k = 1, 2, \dots, K$.
2. Ajustar un modelo GaussianMixture para cada k .

Procedimiento práctico para elegir k

1. Elegir un rango de valores candidatos: $k = 1, 2, \dots, K$.
2. Ajustar un modelo GaussianMixture para cada k .
3. Calcular BIC y/o AIC para cada modelo.

Procedimiento práctico para elegir k

1. Elegir un rango de valores candidatos: $k = 1, 2, \dots, K$.
2. Ajustar un modelo GaussianMixture para cada k .
3. Calcular BIC y/o AIC para cada modelo.
4. Graficar los criterios contra k .

Procedimiento práctico para elegir k

1. Elegir un rango de valores candidatos: $k = 1, 2, \dots, K$.
2. Ajustar un modelo GaussianMixture para cada k .
3. Calcular BIC y/o AIC para cada modelo.
4. Graficar los criterios contra k .
5. Seleccionar el valor de k donde el criterio sea mínimo.

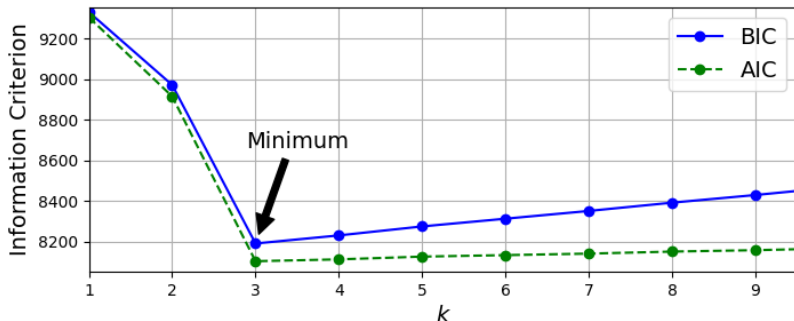
Procedimiento práctico para elegir k

1. Elegir un rango de valores candidatos: $k = 1, 2, \dots, K$.
2. Ajustar un modelo GaussianMixture para cada k .
3. Calcular BIC y/o AIC para cada modelo.
4. Graficar los criterios contra k .
5. Seleccionar el valor de k donde el criterio sea mínimo.

Interpretación

El mínimo indica el mejor compromiso entre ajuste y complejidad.

Ejemplo



Si ambos criterios son mínimos en $k = 3$, entonces $k = 3$ es probablemente una buena elección.

Complejidad computacional: clustering

Sea:

- m : número de instancias,
- n : número de dimensiones,
- k : número de componentes,
- l : número de iteraciones.

Algoritmo	Complejidad	Comentario
K-means	$O(l \times kmn)$	Muy rápido en la práctica
DBSCAN	$O(m^2n)$	No escala bien a datasets grandes
GMM: spherical, diag	$O(l \times kmn)$	Covarianzas simples
GMM: tied, full	$O(l \times (kmn^2 + kn^3))$	Covarianzas más flexibles

Idea: mientras más flexible sea la forma del cluster, más parámetros hay que estimar.

K-means $\approx$ centros

GMM full $\approx$ centros + formas + orientaciones + pesos

¡Gracias!